



Università degli Studi di Padova

Laurea: Informatica

Corso: Ingegneria del Software

Anno Accademico: 2025/26



Gruppo 17

Nome: BitByBit

Email: swe.bitbybit@gmail.com

Verbale Riunione Esterna Numero 3

Stato:	Completato
Versione:	1.0.0
Redattori:	Giovanni Visentin
Verificatori:	Dennis Parolin, Ferdinando Fracasso
Responsabile:	Giovanni Visentin
Destinatari:	Prof. Tullio Vardanega Prof. Cardin Miriade

Indice

1	Informazioni generali	3
2	Ordine del Giorno	4
3	Discussioni	4
4	Decisioni	5
5	To Do	5



1. Informazioni generali

Data:	14 aprile 2026
Ora Inizio:	9:10
Durata:	30 min
Luogo:	Google Meet
Presenti:	Fracasso Ferdinando Manisi Riccardo Parolin Dennis Scaggiante Gabriele Visentin Giovanni
Docenti:	Prof. Cardin
Assenti:	Sanguin Marco



2. Ordine del Giorno

- Analisi della distinzione tra pattern di **Deployment^G Serverless^G** e Microservizi.
- Valutazione della granularità delle Lambda e vincoli di persistenza.
- Revisione dell'**Architettura^G Esagonale**: gestione di Business Objects e DTO.
- Definizione formale dello stereotipo UML <<module>>.

3. Discussioni

- **Architettura^G Backend^G e Pattern**: È stata discussa la differenza tra **Architettura^G** a microservizi e approccio **Serverless^G**. Il Prof. Cardin ha chiarito che non si può definire "microservizio" un'**Architettura^G** in cui più Lambda condividono lo stesso database. Attualmente il sistema è orientato verso un "**Serverless^G** che tende ai microservizi", ma il consiglio principale è quello di spostarsi decisamente verso uno dei due modelli: adottare un pattern pienamente Serverless, assegnando una singola operazione a ciascuna Lambda, oppure passare a un'architettura a microservizi, accorpendo le Lambda per funzionalità logica e garantendo l'indipendenza del database.
- **Dependency Injection in Flutter**: È stato confermato che l'utilizzo del **Framework^G Provider** in Flutter rappresenta la scelta architetturale più corretta. L'uso di Singleton è sconsigliato in quanto crea dipendenze implicite non evidenti dal costruttore e lega il **Ciclo di vita^G** all'intera app, impedendone la ricreazione flessibile.
- **Architettura^G Esagonale**: Si è discusso del ruolo di adapter e interfacce. È emerso che la **Business Logic^G** non deve mai dipendere da dati o risorse esterne e deve operare unicamente utilizzando le classi di dominio. L'impiego di Inbound Adapter (come un ipotetico *SesEventAdapter*) per decodificare gli input esterni e convertirli in oggetti di dominio è un approccio validato e incoraggiato.
- **Gestione Dati Statici**: Per il recupero di dati statici che non necessitano di modifiche da parte dell'**Utente^G** (come la posizione dei luoghi sicuri), è stato sconsigliato l'utilizzo di DynamoDB. L'approccio raccomandato è quello di archiviare tali risorse all'interno di Amazon S3, in quanto risulta una soluzione decisamente più efficiente e meno costosa rispetto alla gestione tramite database NoSQL per dati che rimangono invariati.



4. Decisioni

Descrizione decisione	Codice decisione
Adottare un pattern architetturale a microservizi, accorpando all'interno di una singola funzione Lambda le diverse operazioni relative a una specifica funzionalità o entità di dominio, garantendo al contempo la segregazione del database per ciascun microservizio.	VE PB 3.1
Utilizzare ufficialmente il pacchetto Provider di Flutter per la Dependency Injection e abbandonare il pattern Singleton.	VE PB 3.2
Isolare strettamente la Business Logic^G dai dati esterni avvalendosi di Inbound Adapter e oggetti di dominio secondo i principi dell' Architettura^G Esagonale.	VE PB 3.3
Migrare la memorizzazione dei dati non modificabili dall' Utente^G su Amazon S3, preferendolo a DynamoDB per ottimizzare i costi e l'efficienza nella gestione di risorse statiche.	VE PB 3.4

5. To Do

Le prossime attività da svolgere a seguito di questa riunione sono:

Task	Codice decisione	N°Issue ^G GitHub
Adottare un pattern architetturale a microservizi, assegnando una singola funzione Lambda a ogni macro-funzionalità (o entità di dominio), assicurando che ogni microservizio mantenga il controllo esclusivo sul proprio database.	VE PB 3.1	nessuna
Ristrutturare i diagrammi UML per riflettere l'adozione dell'architettura a microservizi e definire chiaramente i confini di responsabilità di ciascuna Lambda in base alla funzionalità gestita.	VE PB 3.1	nessuna